Day 7 Revision: The Complete Production AI Stack

1. Core integrated summary

The most important revision point is:

> **Vanilla RAG, LlamaIndex, LangChain, LangGraph, and MCP are not five competing solutions. They solve different problems at different layers.**

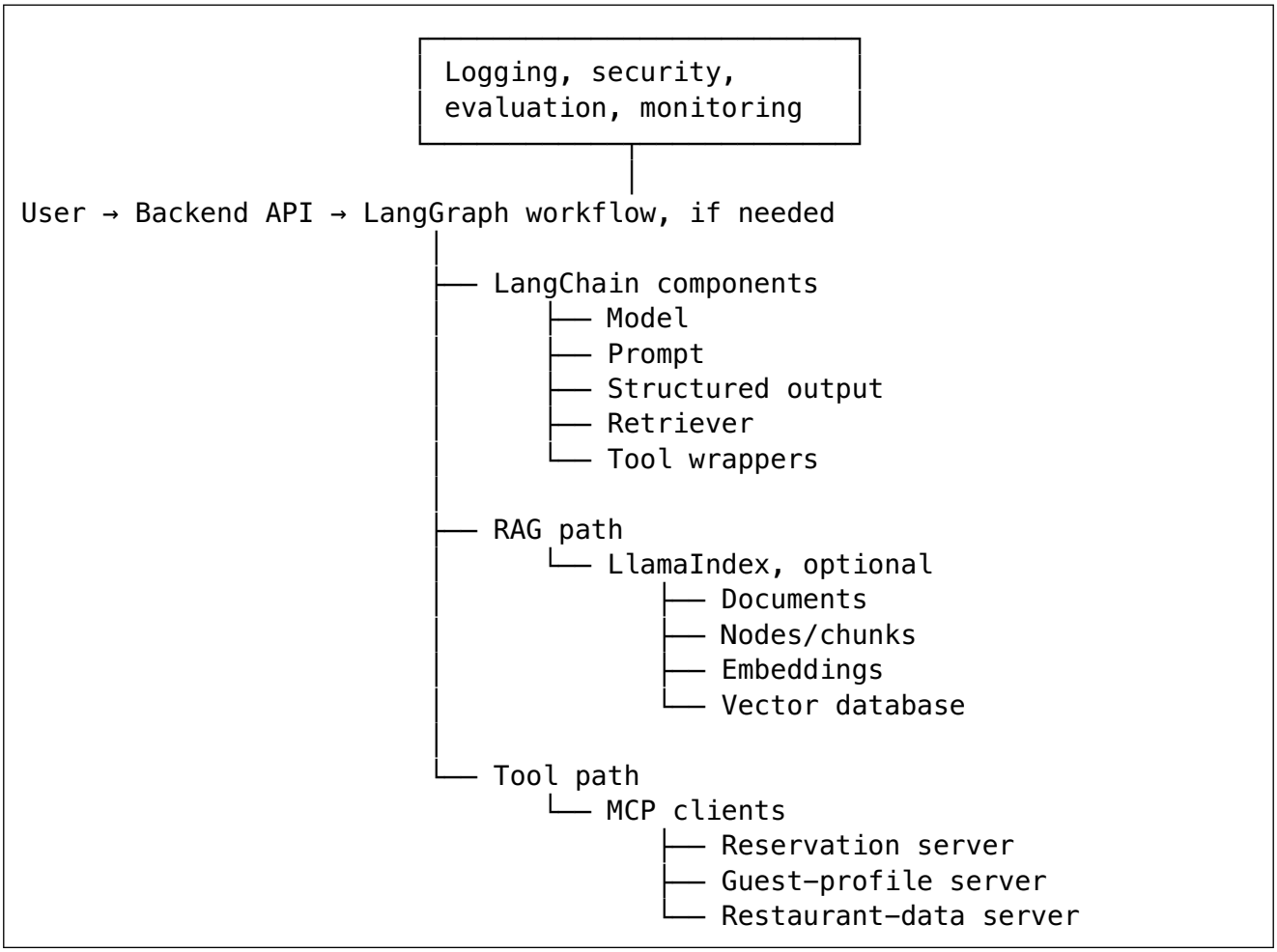
A simple mental model:

Topic	Simple meaning	Main responsibility
Vanilla RAG	A recipe	Find relevant knowledge before asking the LLM to answer
LlamaIndex	A data-focused toolkit	Ingest, parse, index, retrieve, and query enterprise data
LangChain	An application assembly toolkit	Connect models, prompts, retrievers, tools, and structured outputs
LangGraph	A workflow execution engine	Control long-running, stateful, branching AI workflows
MCP	A connectivity standard	Standardize how AI applications connect to external tools and data

A production application may use:

- Only Vanilla RAG implemented with normal Python.
- Vanilla RAG plus LlamaIndex.
- LangChain without LlamaIndex.
- LangGraph with plain Python nodes.
- MCP without LangChain or LangGraph.
- All five, when the system is sufficiently complex.

Complete stack mental model



The official documentation currently describes LlamaIndex as a framework for context-augmented applications and agents over enterprise data, LangChain as an agent/application framework with model and tool integrations, and LangGraph as the lower-level orchestration runtime for durable execution, persistence, and human control. MCP is defined as an open standard connecting AI applications to external systems. (Developer Documentation)

---

2. Foundational category map

2.1 What is a design pattern?

A **design pattern** is a reusable way of solving a common problem.

It describes:

- What steps normally exist.
- How those steps connect.
- What decisions must be made.

It does not force you to use a particular library.

For example, RAG says:

```
Question
   ↓
Retrieve relevant information
   ↓
Give information to LLM
   ↓
Generate grounded answer
```

You can implement this using:

- Plain Python.
- LlamaIndex.
- LangChain.
- Your own internal libraries.
- Cloud-provider services.

Therefore:

> **RAG is primarily an architectural pattern, not a specific framework.**

2.2 What is a framework?

A **framework** is reusable software that provides ready-made components, interfaces, and integrations.

Instead of manually writing everything, a framework may provide:

- Document loaders.
- Prompt templates.
- Retriever interfaces.
- Model integrations.
- Tool definitions.
- Output validation.
- Logging hooks.

LlamaIndex and LangChain are frameworks, although their strongest areas differ.

2.3 What is an orchestration runtime?

Orchestration means controlling multiple steps.

A **runtime** is the machinery that actually executes those steps.

An orchestration runtime answers questions such as:

- Which step should run next?
- What state should be saved?
- What happens if a step fails?
- Should the system retry?
- Should it wait for human approval?
- Can execution resume tomorrow?

LangGraph is mainly used for this responsibility.

2.4 What is a protocol?

A **protocol** is an agreed communication contract.

It defines how two independent systems communicate, including:

- Message structure.
- Request and response format.
- Available capabilities.
- Error representation.
- Connection lifecycle.

HTTP is a general protocol. MCP is a protocol designed for AI applications connecting to tools, data, and reusable context.

2.5 Why are these categories confused?

They overlap at their edges.

For example:

- LlamaIndex provides workflows, not only retrieval.
- LangChain provides retrieval components, not only model calls.
- LangGraph can run tools, not only model steps.
- MCP can expose resources that later participate in RAG.
- A single framework can internally implement several design patterns.

The correct question is therefore not:

> "Which one is the winner?"

The correct question is:

> "Which responsibility am I trying to solve?"

2.6 Category map

Layer	Main question
Vanilla RAG	How do I give the model relevant external knowledge?
LlamaIndex	How do I prepare, organize, retrieve, and query my data?
LangChain	How do I connect and compose the main LLM application components?
LangGraph	How do I reliably execute a multi-step, stateful process?
MCP	How do independent AI applications and external systems communicate through a standard interface?

---

3. Topic-by-topic revision: Days 1–6

Throughout this revision, we will use one hypothetical Disney-like system:

Consistent example: Disney Guest Trip Assistant

A guest asks:

> "My daughter has a nut allergy. Find a suitable restaurant near our park, explain the allergy policy, check availability tomorrow at 7:00 PM for four people, and reserve it after I approve."

This request requires three separate capabilities:

1. **Knowledge:** Read restaurant menus and allergy policies.
2. **Action:** Check availability and create a reservation.
3. **Workflow:** Perform the steps in the correct order and ask for confirmation before booking.

Day 1: Vanilla RAG end to end

3.1 What problem does RAG solve?

An LLM learns general knowledge during training, but that knowledge may be:

- Outdated.
- Incomplete.
- Incorrect.
- Missing private company information.
- Unable to provide reliable source references.

The original RAG approach combined the model's learned knowledge with information retrieved from an external index. This helps applications use updateable external knowledge and provide evidence for answers. (arXiv)

For our example, the LLM may understand food allergies generally, but it does not automatically know:

- The latest restaurant menu.
- The current company allergy policy.
- Restaurant-specific preparation guidance.
- The latest cancellation rules.

RAG retrieves this information before generating the answer.

3.2 Core RAG terms

Document

A **document** is a source of knowledge.

Examples:

- Allergy-policy PDF.
- Restaurant menu.

- Guest-services guide.
- Cancellation-policy webpage.

Chunk

A **chunk** is a smaller piece of a document.

Instead of placing a 100-page guide into the prompt, we might split it into sections such as:

```
Chunk 1: General allergy policy
Chunk 2: Restaurant staff procedures
Chunk 3: Cross-contact warning
Chunk 4: Guest responsibilities
```

Embedding

An **embedding** is a list of numbers representing the meaning of text.

Texts with similar meanings should have embeddings located close to each other.

```
"nut-free dining guidance"
         ≈
"food allergy restaurant procedures"
```

Vector database

A **vector database** stores embeddings and searches for similar embeddings.

Examples include Pinecone, Qdrant, pgvector, Milvus, and Elasticsearch vector search.

Retrieval

Retrieval means finding the most relevant chunks for a question.

Reranking

Reranking means taking the initially retrieved results and sorting them again using a stronger relevance model.

Grounding

Grounding means requiring the answer to be supported by the supplied evidence.

A grounded system should say:

> "The available policy states X."

It should not invent unsupported claims.

3.3 RAG ingestion flow

Ingestion means preparing source data before users ask questions.

```
Documents
   ↓
Parse and clean
   ↓
Split into chunks
   ↓
Attach metadata
   ↓
Create embeddings
   ↓
Store in vector database
```

Example metadata:

```
{
  "document_type": "allergy_policy",
  "restaurant": "Example Restaurant",
  "park": "Example Park",
  "language": "en",
  "effective_date": "2026-06-01",
  "access_level": "public",
  "document_version": "v4"
}
```

Metadata is important because similarity alone may retrieve the wrong:

- Restaurant.
- Language.
- Region.
- Policy version.
- Guest-access level.

3.4 Query flow

```
Guest question
   ↓
Rewrite or normalize question
   ↓
Create query embedding
   ↓
Retrieve candidate chunks
   ↓
Apply metadata filters
   ↓
Rerank candidates
   ↓
Assemble best evidence
   ↓
Construct prompt
   ↓
LLM generates answer
   ↓
Validate citations and grounding
```

3.5 Keyword, semantic, and hybrid retrieval

Keyword retrieval

Finds exact or closely related words.

Good for:

- Restaurant names.
- Policy codes.
- Product names.
- Exact phrases.

Example:

```
"ALGY-204 policy"
```

Semantic retrieval

Finds similar meaning using embeddings.

Good for natural-language questions:

```
"Can the kitchen safely handle my child's allergy?"
```

This may match:

```
"Guests with dietary restrictions should speak with a chef..."
```

even though the words differ.

Hybrid retrieval

Combines keyword and semantic retrieval.

For enterprise systems, hybrid retrieval is often a strong starting point because it handles both:

- Exact business terms.
- Natural-language meaning.

3.6 Chunk size and overlap

Chunk size

A chunk must be:

- Large enough to preserve meaning.
- Small enough to retrieve precisely.

Very small chunks lose context.

Very large chunks add noise and consume tokens.

Overlap

Overlap means repeating some text between neighboring chunks.

```
Chunk 1: paragraphs 1–4
Chunk 2: paragraphs 4–7
```

This helps when an important idea crosses a chunk boundary.

Too much overlap causes:

- Duplicate retrieval.
- Extra storage.
- Higher token usage.
- Repeated context.

3.7 Top-k

Top-k means the number of results initially retrieved.

For example:

```
top-k = 10
```

does not automatically mean all ten results should enter the prompt.

A production pipeline might:

```
Retrieve 20
   ↓
Filter
   ↓
Rerank
   ↓
Pass best 4 to model
```

A high top-k can improve recall but may increase noise, latency, and cost.

3.8 Context assembly

Context assembly means preparing the final evidence placed in the model prompt.

Good context assembly should:

- Remove duplicate chunks.
- Preserve source names.
- Keep related sections together.
- Respect token limits.
- Prefer recent authoritative documents.
- Exclude unauthorized documents.

Example:

```
SOURCE 1: Allergy Policy, version 4
Relevant text: ...

SOURCE 2: Restaurant Menu, July 2026
Relevant text: ...

SOURCE 3: Cancellation Policy
Relevant text: ...
```

3.9 RAG prompt construction

A useful RAG prompt contains:

1. The model's role.
2. The user's question.
3. Retrieved evidence.
4. Clear answering rules.
5. Required output structure.

Example:

```
You are a guest-support assistant.

Answer only from the supplied sources.
Do not claim that any restaurant is completely allergy-free.
Clearly distinguish documented facts from recommendations.
Cite each important statement.
If evidence is missing, say what could not be verified.

Question:
{question}

Sources:
{retrieved_context}
```

3.10 Citation-aware answering

A citation-aware system preserves source information throughout the pipeline.

```
Document
   ↓
Chunk
   ↓
Source ID
   ↓
Retrieved evidence
   ↓
Generated claim
   ↓
Citation
```

Do not generate citations only from the model's memory. The application should maintain the mapping between retrieved chunks and their real source identifiers.

3.11 Retrieval quality versus answer quality

These are different.

Retrieval quality

Did we find the correct evidence?

Useful measurements:

- Recall@k.
- Precision@k.
- Mean Reciprocal Rank.
- Reranker relevance.
- Metadata-filter accuracy.

Answer quality

Did the model correctly use the evidence?

Useful measurements:

- Faithfulness.
- Correctness.
- Citation accuracy.
- Completeness.
- Safety.
- Instruction following.

You can have:

Retrieval	Generation	Result
Good	Good	Correct answer
Good	Bad	Model ignores or misuses evidence
Bad	Good	Well-written answer based on wrong evidence
Bad	Bad	Complete failure

A common mistake is changing the LLM when the actual problem is poor retrieval.

3.12 Production RAG challenges

- Bad document parsing.
- Tables separated from headers.
- Poor chunk boundaries.
- Missing metadata.
- Stale documents.
- Duplicate documents.
- Wrong tenant’s data retrieved.
- Weak query understanding.
- Low recall.
- Too much irrelevant context.
- Unsupported citations.
- Prompt injection inside documents.
- Expensive embedding and reranking calls.

3.13 RAG optimization strategies

- Improve parsing before changing the model.
- Use structure-aware chunking.
- Add useful metadata.
- Combine keyword and semantic search.
- Use query rewriting carefully.
- Retrieve broadly and rerank narrowly.
- Remove duplicate evidence.
- Apply document authority and freshness rules.
- Cache stable retrieval results.

- Evaluate retrieval separately from generation.
- Refuse to answer when supporting evidence is insufficient.

Day 2: LlamaIndex end to end

3.14 What is LlamaIndex?

LlamaIndex is a framework focused strongly on making private or application-specific data usable by LLM applications.

Its official documentation describes facilities for:

- Data connectors.
- Document ingestion.
- Parsing.
- Indexing.
- Query engines.
- Retrieval.
- Response synthesis.
- Agents.
- Event-driven workflows.
- Evaluation and observability. (Developer Documentation)

3.15 Plain RAG versus LlamaIndex

Vanilla RAG is the architecture:

```
Retrieve → augment prompt → generate
```

LlamaIndex provides software components to implement that architecture.

Without LlamaIndex, you manually build:

- Loaders.
- Parsers.
- Chunk models.
- Metadata handling.
- Vector-store integration.
- Retriever logic.
- Query engine.
- Response synthesis.

With LlamaIndex, you can use existing abstractions for many of these responsibilities.

Therefore:

> **LlamaIndex does not replace RAG. It helps implement and extend RAG.**

3.16 Documents and nodes

In LlamaIndex:

- A **Document** normally represents an original source.
- A **Node** normally represents a smaller unit derived from that source.

A node may contain:

```
Text
Metadata
Source relationship
Previous/next relationships
Embedding
Unique identifier
```

You can think of a node as a richer chunk.

3.17 LlamaIndex ingestion path

```
Restaurant PDF
   ↓
Reader or connector
   ↓
Document
   ↓
Parser or transformation
   ↓
Nodes
   ↓
Metadata enrichment
   ↓
Embeddings
   ↓
Index/vector store
```

An **ingestion pipeline** is a repeatable set of transformations applied to incoming data.

Possible transformations:

- Clean text.
- Split documents.
- Extract titles.
- Detect restaurant names.
- Add dates.
- Create embeddings.
- Remove duplicates.

3.18 Index

An **index** is a structure that makes data easier to search.

A vector-store index normally connects nodes to vector-search storage.

The index is not necessarily the database itself. It is often the application abstraction controlling how information is represented and retrieved.

3.19 Retriever

The retriever receives a query and returns relevant nodes.

It may support:

- Vector similarity.
- Keyword search.
- Hybrid search.
- Metadata filters.
- Recursive retrieval.
- Router-based retrieval.
- Fusion of multiple retrievers.
- Reranking.

Example:

```
Question: "What guidance applies to nut allergies?"

Metadata filter:
document_type = allergy_policy
effective_date = latest
region = requested_region
```

3.20 Query engine

A **query engine** combines multiple steps behind one interface:

```
Question
   ↓
Retriever
   ↓
Relevant nodes
   ↓
Post-processing
   ↓
Response synthesizer
   ↓
Answer
```

It is more than vector search. It handles the complete question-answering flow over the indexed data.

3.21 Node postprocessor

A **node postprocessor** changes retrieved nodes before they reach the model.

It may:

- Rerank them.
- Remove low-score nodes.
- Filter old policies.
- Replace chunks with larger parent sections.
- Remove duplicate information.

3.22 Response synthesis

Response synthesis means combining evidence from retrieved nodes into a final answer.

Possible approaches:

- Put all selected nodes in one prompt.
- Summarize nodes individually and combine summaries.
- Incrementally refine an answer.
- Produce structured output.
- Return direct source content without generation.

3.23 LlamaIndex workflows and agents

LlamaIndex also supports:

- Tools.
- Agents.
- Multi-step event-driven workflows.
- Human input.
- Structured extraction.
- Document-oriented agents.

This means its boundary overlaps with LangChain and LangGraph.

However, its strongest mental identity remains:

> **Building context-aware applications over your data.**

LlamaIndex's documentation describes workflows as multi-step processes combining agents, data connectors, RAG sources, and tools. (Developer Documentation)

3.24 Production challenges in LlamaIndex

- Using high-level defaults without understanding them.
- Framework version changes.
- Hidden model or embedding calls.
- Difficult debugging across abstractions.
- Incorrect storage persistence.
- Duplicate ingestion.
- Node-ID instability.
- Index and source-document synchronization.
- Excessive framework coupling.

- Confusion between LlamaIndex workflow state and broader business workflow state.

3.25 LlamaIndex optimization strategies

- Customize chunking for the document type.
- Use deterministic node IDs.
- Track document versions and checksums.
- Make ingestion idempotent.
- Explicitly configure embedding models.
- Separate ingestion services from query services.
- Evaluate retrievers independently.
- Trace query-engine steps.
- Use metadata filters before expensive reranking.
- Keep domain logic outside framework-specific classes where possible.

3.26 When is LlamaIndex a good fit?

Use it when:

- The application is heavily data-oriented.
- You have many document types.
- Parsing and ingestion are significant problems.
- You need flexible retrieval or query engines.
- You want document-focused agents.
- You need to move beyond a basic RAG prototype quickly.

It may be unnecessary when:

- You have ten small documents.
- A single database query solves the problem.
- You already have a mature internal search platform.
- Plain Python plus a vector database is simpler.

Day 3: LangChain end to end

3.27 What is LangChain?

LangChain provides common interfaces and integrations for building LLM applications.

Its current documentation emphasizes:

- Model integrations.
- Tools.
- Agent construction.
- Structured output.
- Middleware.
- Retrieval integrations.
- Portability across model providers.

Its higher-level agents currently run on top of LangGraph. (Docs by LangChain)

A useful study mental model is:

> **LangChain helps assemble the parts around an LLM.**

3.28 Core building blocks

Model

A **model component** provides a common way to call an LLM.

Instead of spreading provider-specific code everywhere:

```
provider_a.generate(...)
provider_b.chat(...)
provider_c.invoke(...)
```

the framework provides a more consistent interface.

This reduces—but does not completely eliminate—provider differences.

Prompt template

A **prompt template** is a reusable prompt with placeholders.

```
You are a restaurant-support assistant.

Question:
{question}

Evidence:
{context}
```

Output parser

An **output parser** converts model text into an application-friendly format.

Example:

```
{
  "restaurant_name": "Example Restaurant",
  "suitable_for_request": true,
  "reason": "...",
  "citations": ["policy-17", "menu-42"]
}
```

Modern structured-output capabilities are often safer than manually parsing arbitrary model text. LangChain supports predictable schema-based results such as JSON objects, Pydantic models, and dataclasses. (Docs by LangChain)

Retriever

A retriever accepts a query and returns documents.

It could be:

- A vector-store retriever.

- A keyword retriever.
- A search API.
- A LlamaIndex-backed retriever.
- An enterprise search system.

Tool

A **tool** is an operation that a model or workflow can request.

Examples:

- `check_restaurant_availability`
- `create_reservation`
- `get_guest_preferences`
- `send_confirmation`

Chain

A **chain** is a fixed composition of steps.

```
Prompt → Model → Output parser
```

or:

```
Retriever → Prompt → Model → Parser
```

A chain is useful when the execution path is mostly predictable.

Integration

An **integration** is an adapter for an external system such as:

- Model provider.
- Vector database.
- Document loader.
- Search service.
- Tool API.
- Observability platform.

3.29 How LangChain helps our example

LangChain might provide:

```
Model interface
    +
RAG prompt template
    +
Retriever adapter
    +
Reservation tool definition
    +
Structured response schema
```

For example, the application could require:

```
{
  "answer": "...",
  "evidence": [],
  "requires_confirmation": true,
  "proposed_action": {
    "type": "create_reservation",
    "restaurant_id": "R-123"
  }
}
```

3.30 Retrieval and tool integration

LangChain can expose both retrieval and APIs as callable components.

But remember:

- A retriever gets knowledge.
- A tool performs or accesses an operation.
- The model may request a tool.
- The application must still validate whether the tool call is allowed.

Do not let the LLM itself become the security boundary.

3.31 Production challenges in LangChain

- Too many abstractions.
- Framework APIs changing.
- Hidden retries or model calls.
- Difficult stack traces.
- Weak tool schemas.
- Parsing failures.
- Provider capabilities not being identical.
- Treating every problem as an agent problem.
- Business logic becoming tightly coupled to framework classes.
- Dependency bloat.

3.32 LangChain optimization strategies

- Use only the components you need.
- Prefer typed structured output.
- Keep prompts versioned.
- Add timeouts and retry limits.
- Wrap provider differences behind your own interfaces.
- Keep authorization outside the LLM.
- Trace every model and tool call.
- Test chains with fixed datasets.
- Avoid deeply nested chains.
- Keep critical business rules in ordinary code.

3.33 When is LangChain useful?

Use it when:

- You need multiple model integrations.
- You need prompt and structured-output abstractions.
- You need ready-made retriever or tool integrations.
- You are building a moderately complex LLM application.
- You want to move quickly without writing every adapter.

It may be unnecessary when:

```
One prompt
   ↓
One model call
   ↓
One response
```

A direct provider SDK may be clearer in that situation.

Day 4: LangGraph end to end

3.34 What is LangGraph?

LangGraph is an orchestration runtime for stateful workflows and agents.

Its core model contains:

- **State:** the shared current information.
- **Nodes:** steps that perform work.
- **Edges:** rules defining what runs next.

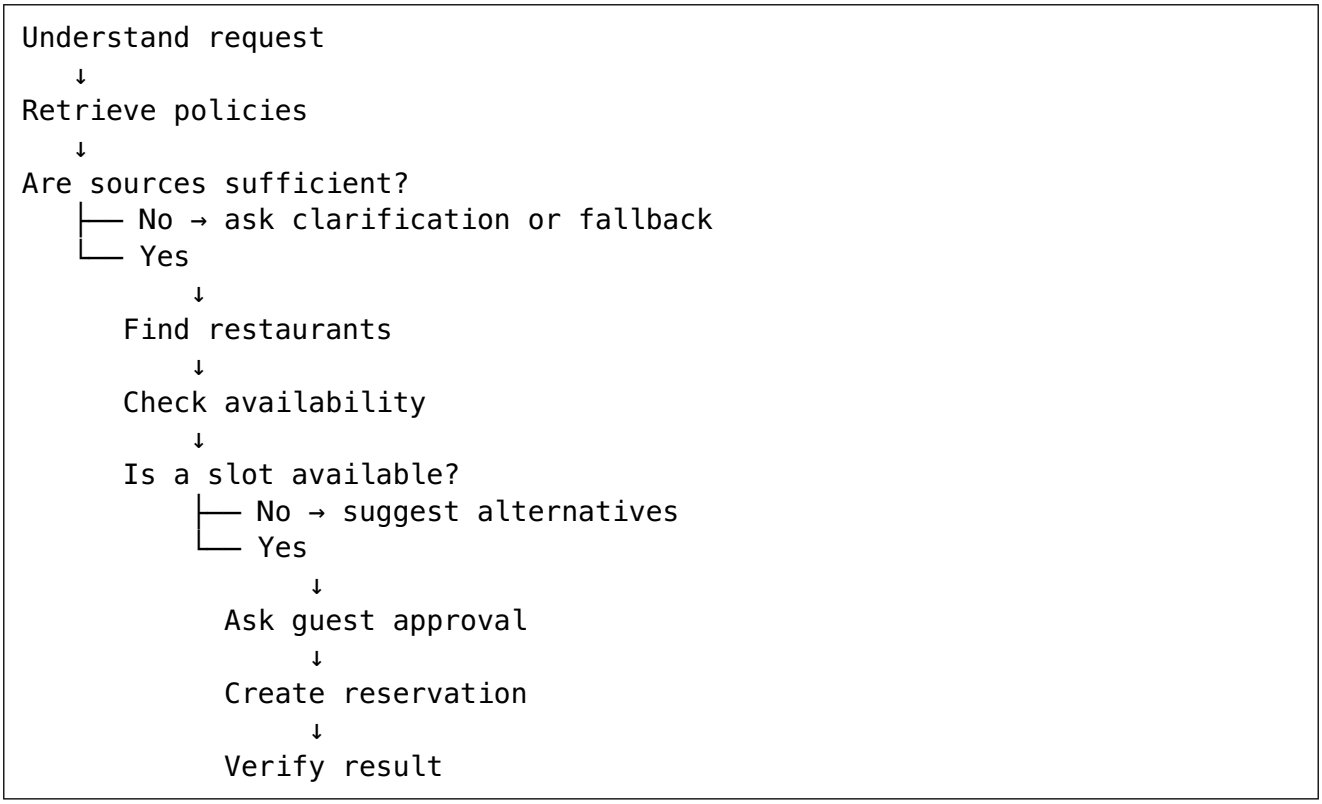
Official documentation describes it as a runtime for durable execution, persistence, streaming, and human-in-the-loop control. (Docs by LangChain)

3.35 Why simple chains are not always enough

A chain is good for:

```
A → B → C
```

Our guest request is more complex:



This contains:

- Branches.
- Tool calls.
- Validation.
- Waiting.
- Retries.
- Human approval.
- Recovery after failure.

That is where graph-based orchestration becomes useful.

3.36 State

State is the shared structured information carried through the workflow.

Example:

```
{
  "request_id": "REQ-701",
  "guest_question": "...",
  "party_size": 4,
  "requested_time": "19:00",
  "allergies": ["nuts"],
  "retrieved_sources": [],
  "candidate_restaurants": [],
  "selected_slot": null,
  "guest_approved": false,
  "reservation_id": null,
  "errors": [],
  "attempt_count": 0
}
```

State is not just chat history. It can include:

- Business identifiers.
- Tool outputs.
- Decisions.
- Validation results.
- Retry counts.
- Approval status.

3.37 Nodes

A node is one unit of work.

Examples:

- `classify_request`
- `retrieve_policy`
- `evaluate_evidence`
- `search_restaurants`
- `check_availability`
- `request_approval`
- `create_reservation`
- `verify_reservation`
- `generate_response`

Keep nodes focused. A node that retrieves, books, verifies, and writes the answer is too large.

3.38 Edges and routing

An edge tells the graph what runs next.

Fixed edge

```
retrieve_policy → evaluate_evidence
```

Conditional edge

```
evaluate_evidence
   ├── sufficient → search_restaurants
   └── insufficient → safe_fallback
```

LangGraph supports fixed and conditional transitions, loops, and state-based routing. (Docs by LangChain)

3.39 Deterministic versus agentic workflow

Deterministic workflow

The developer controls the path.

```
Retrieve → validate → check → approve → book
```

Best for:

- Payments.
- Reservations.
- Account changes.
- Safety-sensitive operations.
- Compliance processes.

Agentic workflow

The model helps decide the next action.

```
Model examines state
   ↓
Chooses search, retrieval, or another tool
   ↓
Observes result
   ↓
Chooses again
```

Best for:

- Open-ended research.
- Complex investigation.
- Tasks where the correct path is not known beforehand.

Production recommendation

Use deterministic control for critical operations and limited agentic reasoning inside safe boundaries.

```
Deterministic outer workflow
          +
Controlled agentic inner steps
```

3.40 Checkpointing

A **checkpoint** is a saved snapshot of workflow state.

After the guest approves the reservation, the system might save:

```
guest_approved = true
selected_slot = 7:00 PM
```

If the reservation API temporarily fails, the workflow can resume without:

- Repeating document retrieval.
- Asking for approval again.
- Losing the selected slot.

LangGraph persistence saves graph state as checkpoints and supports resumption, human review, fault recovery, and historical debugging. (Docs by LangChain)

3.41 Durable execution

Durable execution means a workflow can survive:

- Process restart.
- Temporary dependency failure.
- Long waiting periods.
- Human approval delays.

It does not mean that every side effect is automatically safe.

You still need **idempotency**.

Idempotency

An idempotent operation can be safely retried without accidentally performing the action twice.

For booking:

```
Idempotency key = request_id + selected_slot + guest_id
```

Without this, a retry could create two reservations.

3.42 Human-in-the-loop

Human-in-the-loop means execution pauses for a person's input.

For our example:

```
Proposed reservation:
Restaurant: Example Restaurant
Time: 7:00 PM
Party size: 4
Cancellation terms: ...

Approve?
```

Only after approval should the booking tool run.

LangGraph interrupts can save state, pause execution, and resume after external input. (Docs by LangChain)

3.43 Workflow design principles

- Keep state explicit and typed.
- Keep nodes small.
- Separate reasoning from side effects.
- Validate before every sensitive action.
- Use maximum-step limits.
- Make side-effecting tools idempotent.
- Put retries around transient failures only.
- Do not retry authorization failures blindly.
- Record routing decisions.
- Add human approval for high-impact operations.
- Design compensation for partially completed work.

3.44 Production challenges in LangGraph

- State becoming too large.
- Hidden state mutations.
- Infinite loops.
- Excessive model calls.
- Duplicate side effects during retries.
- Migrating graph definitions while runs are paused.
- Debugging parallel paths.
- Complex checkpoint storage.
- Unclear ownership of failed workflows.
- Too much agent freedom.

3.45 LangGraph optimization strategies

- Use typed minimal state.
- Store large artifacts outside the graph and keep references.
- Limit loop iterations.
- Route using normal code when possible.
- Use smaller models for simple decisions.
- Parallelize only independent nodes.
- Cache repeatable read-only steps.

- Use durable production checkpoint storage.
- Add per-node latency and cost metrics.
- Test failure and resume paths.
- Version state schemas and graph definitions.

3.46 When is LangGraph the right choice?

Use it when you need several of these:

- Multiple steps.
- Conditional routing.
- Loops.
- Persistent state.
- Human approval.
- Long-running execution.
- Failure recovery.
- Deterministic plus agentic behavior.
- Auditable workflow decisions.

Do not add it merely because the application calls two functions.

Day 5: MCP end to end

3.47 What is MCP?

MCP stands for **Model Context Protocol**.

It is a standard way for AI applications to connect to external:

- Tools.
- Data sources.
- Reusable resources.
- Prompt templates and workflows.

The official documentation compares MCP to a common connector for AI applications, allowing them to access data sources, tools, and workflows through a standardized interface. (Model Context Protocol)

3.48 What problem does MCP solve?

Without a standard:

```
Application A → custom Salesforce adapter
Application A → custom database adapter
Application A → custom reservation adapter

Application B → another Salesforce adapter
Application B → another database adapter
Application B → another reservation adapter
```

This creates repeated work and inconsistent behavior.

With MCP:

```
AI Application A ─┐
AI Application B ─┼── MCP interface → Reservation MCP server
AI Application C ─┘
```

The server exposes capabilities using one common protocol.

MCP reduces integration duplication, but it does not remove the need to implement the underlying business API.

3.49 Host, client, and server

Host

The **host** is the main AI application.

For our example:

```
Disney-like Guest Assistant backend
```

The host controls:

- Which MCP servers may connect.
- What context may be shared.
- User consent.
- Security policy.
- Model interaction.

Client

An **MCP client** lives inside the host and maintains a connection to one MCP server.

Conceptually:

```
Host
 ├── Dining MCP client
 ├── Guest-profile MCP client
 └── Policy MCP client
```

Server

An **MCP server** exposes capabilities.

Examples:

```
Dining server
  ├── search_restaurants
  ├── check_availability
  └── create_reservation

Policy server
  ├── allergy-policy resource
  └── cancellation-policy resource
```

MCP uses a client-host-server architecture designed to separate responsibilities and security boundaries. (Model Context Protocol)

3.50 Tools, resources, and prompts

Tool

A tool performs an operation.

```
check_availability(date, time, party_size)
```

A tool may read data or cause a side effect.

Resource

A resource exposes data or content.

Examples:

```
policy://allergies/latest
restaurant://R-123/menu
guest://preferences
```

The host decides how to use this data:

- Pass it to the model.
- Index it for RAG.
- Filter it.
- Display it directly.

MCP resources provide structured access to data, while tools expose invokable operations. (Model Context Protocol)

Prompt

An MCP server can expose reusable prompt templates or workflows.

For example:

```
"Summarize restaurant allergy guidance"
```

Prompts are useful but generally less central than tools and resources in many backend architectures.

3.51 Stateful session mental model

MCP is session-oriented.

A simple mental model:

```
Connect
   ↓
Negotiate supported capabilities
   ↓
Discover tools/resources
   ↓
Exchange requests and responses
   ↓
Handle notifications and updates
   ↓
Close connection
```

A stateful MCP session does **not** automatically mean:

- Permanent customer memory.
- Business-workflow state.
- LangGraph checkpoint state.
- Conversation history forever.

It means the client and server maintain communication context during their protocol session.

3.52 JSON-RPC in simple language

MCP messages use JSON-RPC.

A request conceptually looks like:

```
{
  "id": 17,
  "method": "tools/call",
  "params": {
    "name": "check_availability",
    "arguments": {
      "date": "tomorrow",
      "party_size": 4
    }
  }
}
```

The response contains either:

```
{
  "id": 17,
  "result": {}
}
```

or:

```
{
  "id": 17,
  "error": {}
}
```

The `id` connects the response to the original request. MCP requires client-server messages to follow JSON-RPC 2.0. (Model Context Protocol)

3.53 MCP versus function calling

Function calling	**MCP**
Model expresses a request to call a function	Application connects to an external MCP server
Often configured inside one application	Standardized across compatible applications
Tools may be hard-coded at startup	Capabilities can be discovered through the protocol
Defines model-facing schemas	Defines client-server communication and capabilities
Does not standardize external server lifecycle	Includes connection and capability negotiation concepts

They can work together:

```
LLM produces tool call
        ↓
Host validates request
        ↓
MCP client invokes MCP server
        ↓
Server calls business API
        ↓
Result returns to host
        ↓
Host gives safe result to model
```

MCP does not replace model tool calling. It can standardize the connection behind the tool.

3.54 Security and governance

Important controls include:

- Authenticate the user.
- Authorize every sensitive capability.
- Use least-privilege scopes.
- Validate tool arguments.
- Require approval for high-impact actions.
- Do not trust tool descriptions blindly.
- Isolate local servers.
- Apply timeouts and resource limits.

- Redact secrets from logs.
- Maintain audit logs.
- Prevent cross-user context leakage.
- Validate server identity.
- Treat returned content as untrusted input.

Current MCP security guidance recommends protected authorization flows, HTTPS, secure token storage, least-privilege scopes, sandboxing where appropriate, and careful session handling. (Model Context Protocol)

3.55 Production MCP challenges

- Untrusted MCP servers.
- Too many exposed tools.
- Ambiguous tool descriptions.
- Schema-version incompatibility.
- Long-running or hanging tool calls.
- Authentication propagation.
- Excessive privileges.
- Prompt injection through returned resources.
- Sensitive-data leakage.
- Tool-name collisions.
- Server availability problems.
- Ownership and support ambiguity.

3.56 MCP optimization strategies

- Expose small, domain-focused servers.
- Use narrow tool schemas.
- Group tools by business capability.
- Apply tool allowlists.
- Load only relevant tools.
- Cache read-only resources where safe.
- Add deadlines and cancellation.
- Use connection pooling where supported.
- Validate structured responses.
- Version schemas.
- Add health checks and tracing.
- Separate read tools from write tools.
- Require confirmation for write operations.
- Keep business authorization inside the underlying service.

3.57 When is MCP worth adopting?

Use MCP when:

- Several AI applications need the same systems.
- Tools should be reusable across teams.
- You need standardized discovery and connectivity.

- Enterprise systems are managed by separate owners.
- You want to reduce framework-specific adapters.
- Tool governance is becoming an organizational problem.

MCP may be unnecessary when:

- One application calls one stable internal API.
- No reuse is expected.
- A direct typed SDK is simpler.
- The integration is entirely internal and already standardized effectively.

Day 6: Full interrelationship

3.58 How Vanilla RAG connects to LlamaIndex

```
Vanilla RAG = the recipe
LlamaIndex = components for implementing the recipe
```

Example:

```
RAG says:
Load → chunk → embed → retrieve → generate

LlamaIndex provides:
Reader → Document → Node parser → Index → Retriever → Query engine
```

3.59 How LlamaIndex connects to LangChain

Both can implement retrieval and connect to models.

A practical separation could be:

```
LlamaIndex:
Own data ingestion, indexing and retrieval

LangChain:
Own prompts, model invocation and structured output
```

For example:

```
LangChain application
   ↓
Calls LlamaIndex-backed retriever
   ↓
Receives relevant nodes/documents
   ↓
Builds prompt
   ↓
Calls model
```

But using both is not mandatory. Either framework may be sufficient alone.

3.60 How LangChain connects to LangGraph

A LangGraph node can use LangChain components.

```
LangGraph node: retrieve_policy
   └── invokes LangChain retriever

LangGraph node: generate_answer
   └── invokes LangChain prompt + model + parser

LangGraph node: decide_next_action
   └── invokes LangChain agent/model
```

Current LangChain agents are themselves built on LangGraph, but developers can use LangGraph directly when they need lower-level control. (Docs by LangChain)

3.61 How LangGraph connects to MCP

LangGraph decides **when** a tool should run.

MCP standardizes **how** the external tool is accessed.

```
LangGraph
  "Run availability check now"
            ↓
MCP client
  "Call dining server using standard protocol"
            ↓
Dining MCP server
  "Call reservation platform"
```

LangGraph owns workflow control.

MCP owns connectivity.

3.62 Where surrounding technologies fit

Embedding model

Converts text into vectors.

Used inside the RAG/LlamaIndex retrieval layer.

Vector database

Stores embeddings and searchable metadata.

It is infrastructure, not an orchestration framework.

LLM provider

Generates answers, classifications, summaries, or tool requests.

It may be accessed directly or through LangChain/LlamaIndex.

Backend API

Handles:

- Authentication.
- Request validation.
- Rate limiting.
- Streaming.
- User sessions.
- API contracts.

FastAPI, Flask, Go, Java, or another backend technology may serve this layer.

Business APIs

Perform real operations:

- Create reservation.
- Read guest profile.
- Send notification.
- Check inventory.

MCP may expose these APIs to AI applications through a standard interface.

3.63 Overlap and difference table

Topic	Mainly for	Can also do	Not mainly for
Vanilla RAG	Grounding answers using retrieved knowledge	Reranking, query rewriting, citations	General workflow orchestration
LlamaIndex	Data ingestion, indexing, retrieval, data-aware agents	Workflows, tools, evaluation	Enterprise-wide connectivity protocol
LangChain	Models, prompts, tools, structured output, application composition	Retrieval and agents	Durable business-process execution by itself
LangGraph	Stateful workflow and agent execution	Tool routing, retries, human approval	Document parsing and indexing
MCP	Standardized tool/data connectivity	Resources and reusable prompts	Deciding business workflow order

---

4. One complete end-to-end production example

Guest request

> “My daughter has a nut allergy. Find a suitable restaurant tomorrow at 7:00 PM for four people, explain the relevant policy, and reserve it after I approve.”

This is a hypothetical Disney-like architecture, not a description of Disney's internal implementation.

Step 1: Request enters the backend API

The API performs:

- Authentication.
- Request validation.
- Rate limiting.
- Request-ID generation.
- User-consent checks.
- Basic safety checks.

```
POST /assistant/requests

request_id = REQ-701
user_id = U-42
```

The raw user message should not directly call a tool.

Step 2: Application determines required capabilities

A router identifies:

```
{
  "needs_retrieval": true,
  "needs_external_tools": true,
  "needs_multistep_workflow": true,
  "requires_human_confirmation": true
}
```

Therefore:

- RAG is needed for policies and menus.
- Tools are needed for live availability and booking.
- LangGraph is useful for multi-step control.
- MCP may standardize access to the reservation systems.

Step 3: LangGraph initializes workflow state

```
{
  "request_id": "REQ-701",
  "user_request": "...",
  "allergy": "nuts",
  "party_size": 4,
  "requested_time": "19:00",
  "policy_evidence": [],
  "restaurants": [],
  "selected_slot": null,
  "approved": false,
  "reservation_id": null
}
```

Step 4: Retrieval path begins

LlamaIndex's possible role

An offline ingestion service has already:

1. Loaded policy PDFs and restaurant menus.
2. Parsed them.
3. Created documents and nodes.
4. Added metadata.
5. Created embeddings.
6. Stored them in a vector database.

At query time, LlamaIndex can:

- Apply restaurant and date filters.
- Retrieve relevant nodes.
- Rerank evidence.
- Return source metadata.

Vanilla RAG role

The overall retrieval pattern remains:

```
Question
   ↓
Retrieve evidence
   ↓
Add evidence to prompt
   ↓
Generate grounded result
```

Step 5: Evidence validation

A deterministic validation node checks:

- Are policies current?

- Are sources authoritative?
- Is there direct evidence?
- Are citations available?
- Are any statements contradictory?

If evidence is weak:

```
Do not continue with a strong recommendation.
Return a qualified response or escalate.
```

Step 6: Candidate restaurants are identified

The application uses retrieved menu and policy information to find candidates.

The LLM may help summarize or classify evidence, but hard filters remain normal code:

```
candidate.active is True
candidate.accepts_reservations is True
candidate.location in permitted_locations
```

The model should not decide access permissions.

Step 7: Check live availability through MCP

LangGraph reaches the `check_availability` node.

```
LangGraph node
    ↓
MCP client
    ↓
Dining MCP server
    ↓
Reservation business API
```

Tool input:

```
{
  "restaurant_ids": ["R-123", "R-456"],
  "date": "2026-07-29",
  "preferred_time": "19:00",
  "party_size": 4
}
```

Tool output:

```
{
  "available_slots": [
    {
      "restaurant_id": "R-123",
      "time": "19:15",
      "slot_token": "temporary-token"
    }
  ]
}
```

Step 8: LangChain assembles the response proposal

LangChain may combine:

- Prompt template.
- Retrieved evidence.
- Tool output.
- Model.
- Structured-output schema.

The result must distinguish:

- Policy facts.
- Menu information.
- Live availability.
- Limitations.
- Required confirmation.

Example structure:

```
{
  "recommendation": "...",
  "policy_summary": "...",
  "available_slot": "19:15",
  "citations": ["POL-17", "MENU-R123-8"],
  "limitations": [
    "No restaurant can be represented as completely free from cross-contact
risk."
  ],
  "requires_confirmation": true
}
```

Step 9: Human approval

LangGraph pauses.

The guest sees:

```
Proposed reservation:
Restaurant: Example Restaurant
Date: July 29, 2026
Time: 7:15 PM
Party size: 4

Important allergy guidance:
...

Cancellation terms:
...

Approve booking?
```

No booking occurs yet.

Step 10: Booking tool executes

After approval:

```
LangGraph resumes
   ↓
Validates slot token
   ↓
Calls create_reservation through MCP
   ↓
Uses idempotency key
   ↓
Receives reservation ID
```

Example:

```
{
  "idempotency_key": "REQ-701-R123-1915",
  "slot_token": "temporary-token",
  "party_size": 4
}
```

Step 11: Verify the result

Never assume a successful HTTP response means the business action completed correctly.

A verification node checks:

- Reservation ID exists.
- Correct restaurant.
- Correct date and time.
- Correct party size.
- Correct guest.
- No duplicate reservation.

Step 12: Generate final response

The final answer includes:

- Reservation confirmation.
- Verified booking details.
- Relevant policy summary.
- Source citations.
- Important safety qualification.
- What the guest should do next.

Step 13: Cross-cutting production controls

Logging

Record:

- Request ID.
- Workflow nodes.
- Retrieval queries.
- Document IDs.
- Tool calls.
- Latency.
- Token usage.
- Retry attempts.
- Final status.

Do not log secrets or unnecessarily sensitive personal information.

Evaluation

Measure:

- Retrieval recall.
- Citation correctness.
- Answer faithfulness.
- Tool-selection accuracy.
- Booking success rate.
- Duplicate-booking rate.
- Human-approval completion rate.
- End-to-end latency.

Fallback

Possible fallbacks:

```
Vector search fails
   → use keyword search

Primary LLM fails
   → use secondary provider or safe template

Availability server fails
   → return knowledge answer without claiming availability

Booking fails after approval
   → preserve state and offer retry

Evidence is insufficient
   → explain limitation instead of guessing
```

Security

- Tenant filters.
- Role-based access.
- Tool allowlists.
- Input validation.
- Output validation.
- User confirmation.
- Audit trails.
- Prompt-injection defenses.
- Secret management.
- Data-retention controls.

---

5. Production-grade challenges across the full stack

Challenge	What can go wrong	Staff-level response
Wrong framework choice	Team adds multiple frameworks to a simple app	Start with requirements, not tool names
Over-engineering	Simple Q&A becomes an agent graph	Use direct code until complexity justifies orchestration
Weak layer boundaries	Retrieval, workflow, and API logic mix together	Assign clear ownership to each layer
Retrieval problems	Wrong evidence reaches the model	Evaluate parsing, retrieval, reranking, and filtering
Workflow complexity	Loops and branches become unmanageable	Use typed state, bounded routes, and small nodes
Tool failures	External APIs time out or return partial results	Add timeouts, retries, circuit breakers, and validation
Duplicate side effects	Retry creates multiple bookings	Use idempotency and verification
Security risks	Agent accesses tools or data it should not see	Enforce authorization outside the model
Weak observability	Team sees only final answer	Trace retrieval, prompts, routing, tools, and state
Weak evaluation	Quality judged using a few demos	Build offline datasets and production metrics
High latency	Sequential model, retrieval, and tool calls accumulate	Parallelize safe steps, cache, and use smaller models
High cost	Too much context and too many model calls	Budget tokens and calls per request
Provider lock-in	Application relies on one provider’s behavior	Use internal interfaces and contract tests
Ownership confusion	Nobody owns failed workflows	Define service, data, workflow, and tool owners

---

6. Optimization strategies across the full stack

6.1 Separate concerns clearly

A healthy architecture might assign:

```
Backend API:
Authentication and external contract

RAG/LlamaIndex:
Knowledge preparation and retrieval

LangChain:
Model, prompt, retriever and tool composition

LangGraph:
Workflow execution and state

MCP:
External capability connectivity

Business services:
Actual business rules and transactions
```

6.2 Use the simplest sufficient layer

```
Simple answer from known input
    → direct model SDK

Simple document Q&A
    → plain RAG

Document-heavy application
    → consider LlamaIndex

Several model/tool components
    → consider LangChain

Stateful branching workflow
    → consider LangGraph

Reusable external tool ecosystem
    → consider MCP
```

Do not start with all five.

6.3 Improve retrieval systematically

Use this order:

```
Parsing
  ↓
Chunking
  ↓
Metadata
  ↓
Initial retrieval
  ↓
Filtering
  ↓
Reranking
  ↓
Context assembly
  ↓
Generation
```

Do not jump directly to changing the LLM.

6.4 Improve workflow control

- Prefer deterministic routes for sensitive operations.
- Limit agent loops.
- Add maximum tool calls.
- Validate every transition.
- Save state before and after important actions.
- Use idempotency.
- Require approval before write operations.

6.5 Improve tool boundaries

Every tool should have:

- Clear name.
- Clear description.
- Narrow input schema.
- Typed output schema.
- Authorization rules.
- Timeout.
- Error contract.
- Idempotency behavior.
- Audit classification.

Bad tool:

```
manage_restaurant_everything(input: string)
```

Better tools:

```
search_restaurants(filters)
check_availability(restaurant_id, date, party_size)
create_reservation(slot_token, guest_id, idempotency_key)
cancel_reservation(reservation_id, reason)
```

6.6 Improve observability

For each request, trace:

```
API request
 → routing decision
 → retrieval query
 → retrieved source IDs
 → reranker scores
 → prompt version
 → model call
 → graph node
 → MCP tool call
 → business API result
 → final response
```

A final answer alone is not enough for debugging.

6.7 Improve evaluation

Create three evaluation layers.

Component evaluation

- Parser quality.
- Retrieval recall.
- Reranker quality.
- Structured-output validity.
- Tool success rate.

Workflow evaluation

- Correct route selected.
- Approval required at correct time.
- Retries bounded.
- Resume behavior correct.
- No duplicate actions.

End-to-end evaluation

- Task completed correctly.
- Answer supported by evidence.
- Safe behavior.
- Acceptable latency and cost.
- Good user experience.

6.8 Improve cost and latency

- Use smaller models for classification and routing.
- Use a larger model only for difficult synthesis.
- Cache embeddings.
- Cache stable policy retrieval.
- Batch ingestion calls.
- Parallelize independent read operations.
- Limit context size.
- Avoid repeatedly passing full chat history.
- Use deterministic code instead of an LLM for straightforward rules.
- Stop workflows as soon as the result is sufficient.

6.9 Improve fallback behavior

A fallback should reduce capability honestly.

Example:

```
Full capability:
Policy answer + availability + reservation

Reservation system unavailable:
Policy answer + explain that live availability could not be checked

Retrieval unavailable:
Do not invent policy details; direct the guest to an authoritative channel
```

A fallback must not pretend the failed dependency succeeded.

---

7. Common misconceptions to avoid

Misconception 1: RAG is a framework

No. RAG is primarily a pattern. Frameworks help implement it.

Misconception 2: LlamaIndex is only a PDF chatbot library

No. It includes ingestion, indexing, retrieval, query engines, agents, workflows, extraction, and evaluation.

Misconception 3: LangChain is required for every LLM call

No. A direct SDK is often better for simple applications.

Misconception 4: LangChain and LangGraph are competitors

Not usually. LangChain provides higher-level application and agent components; LangGraph provides lower-level stateful orchestration.

Misconception 5: LangGraph automatically makes an agent reliable

No. Reliability still requires:

- Good state design.
- Validation.
- Idempotency.
- Error handling.
- Evaluation.
- Secure tools.

Misconception 6: MCP is another agent framework

No. MCP is a connectivity protocol. It does not decide the workflow.

Misconception 7: MCP replaces REST APIs

No. An MCP server often calls REST, gRPC, database, or internal SDK interfaces behind the scenes.

Misconception 8: Function calling and MCP are the same

No. Function calling communicates model intent to the host. MCP standardizes communication between the host and external capability servers.

Misconception 9: More retrieved chunks always improve RAG

No. More chunks can create noise, contradiction, cost, and latency.

Misconception 10: A citation means the answer is correct

No. The citation must actually support the claim.

Misconception 11: An agent should control all decisions

No. Sensitive business operations need deterministic rules and authorization.

Misconception 12: Using all frameworks creates a production-grade system

No. Production quality comes from clear architecture, testing, security, evaluation, observability, and ownership.

---

8. Staff-level interview angle

8.1 How to explain all five clearly

A strong concise explanation:

> “RAG is a design pattern for grounding model responses with retrieved external knowledge. LlamaIndex is a data-oriented framework that helps with ingestion, parsing, indexing, retrieval, and data-aware query workflows. LangChain provides reusable interfaces and integrations for models, prompts, retrievers, tools, and structured outputs. LangGraph is a lower-level runtime for stateful, branching, durable workflows with retries and human approval. MCP is a protocol

> that standardizes how an AI host connects to external tool and data servers. They operate at different layers and can be used independently or together."

8.2 Answering "When would you use each one?"

Vanilla RAG

> "I would use RAG when the answer must depend on private, current, or source-backed knowledge that is not reliably present in the model."

LlamaIndex

> "I would use LlamaIndex when data ingestion, document parsing, indexing, metadata, retrieval, or complex querying over enterprise data is a major part of the problem."

LangChain

> "I would use LangChain when I need reusable model, prompt, retriever, tool, agent, or structured-output integrations and those abstractions reduce application glue code."

LangGraph

> "I would use LangGraph when the application has stateful multi-step execution, branching, loops, long-running tasks, recovery, or human approval. I would not use it for a simple linear model call."

MCP

> "I would use MCP when multiple AI applications or teams need reusable, standardized access to external tools and data. For one small integration, a direct API client may be simpler."

8.3 How to justify trade-offs

A Staff Engineer should not only describe capabilities. They should explain costs.

Choice	Benefit	Cost
Plain RAG	Simple and controllable	More custom engineering
LlamaIndex	Strong data abstractions	Dependency and abstraction complexity
LangChain	Broad integrations	Framework coupling and debugging complexity
LangGraph	Explicit control and recovery	Additional workflow and state infrastructure
MCP	Reuse and interoperability	Server governance, security, and operational overhead

A strong interview statement:

> "I introduce an abstraction only when its operational benefit is greater than its complexity cost."

8.4 How to explain the architecture strongly

Use responsibility-based language:

> "The API layer authenticates the guest and validates the request. The retrieval layer provides evidence from approved policies and restaurant content. The model layer performs controlled classification and synthesis. The workflow layer manages state, branches, retries, and approval. The connectivity layer provides governed access to live reservation capabilities. The business service remains the final authority for permissions and transactions. Observability and evaluation cover every layer."

This sounds stronger than simply listing framework names.

8.5 Disney-like production mapping

A Disney-like enterprise may have AI use cases involving:

- Guest support.
- Park and resort information.
- Dining assistance.
- Content discovery.
- Employee knowledge.
- Operational incident analysis.
- Media metadata.
- Customer-service workflows.

At Staff level, focus on:

- High traffic.
- Availability.
- Guest safety.
- Internationalization.
- Data access control.
- Current information.
- Human escalation.
- Auditable actions.
- Cost management.
- Graceful degradation.

The important architecture principle is:

```
LLM = reasoning and language component

LLM ≠ source of truth
LLM ≠ authorization engine
LLM ≠ transaction system
LLM ≠ workflow database
```

Final revision memory map

```
RAG
"Give the model relevant knowledge."

LlamaIndex
"Prepare and retrieve knowledge from data."

LangChain
"Connect and assemble LLM application components."

LangGraph
"Control stateful multi-step execution."

MCP
"Standardize connections to external capabilities."
```

And the complete production flow:

```
User request
    ↓
Secure backend API
    ↓
Decide: direct answer, retrieval, tool, or workflow?
    ↓
LangGraph controls the process, when complexity requires it
    ↓
LangChain may compose models, prompts, retrievers, and tools
    ↓
LlamaIndex may manage data ingestion and retrieval
    ↓
Vanilla RAG pattern grounds the answer in evidence
    ↓
MCP may connect to external tools and enterprise data
    ↓
Validate evidence and tool results
    ↓
Request human approval for sensitive action
    ↓
Execute and verify the business operation
    ↓
Return a cited, safe, observable response
```

> **Staff-level conclusion:** Begin with the business problem and reliability requirements. Select the smallest set of layers that satisfies them. Keep retrieval, reasoning, orchestration, connectivity, authorization, and business transactions as separate responsibilities.